

---

# Big Data

## Management and Analytics

TU Clausthal, Institut für Informatik

Summer Semester 2026

---

**Due Date: 27 May 2026, 16:00**

### Instructions — Please Read Carefully

In this assignment, we will use Neo4j again and practise role management and database security. For the exercises, we will use the “E-Commerce”-DB.

**Cypher Reference:** <https://neo4j.com/docs/cypher-refcard/current/>

Include all essential ideas and steps necessary to derive a solution. Write your solution in an understandable form or draw the structure with an appropriate tool. If there is missing information, make an assumption and state it explicitly — it will hold for the entire assignment unless declared otherwise.

### Assignment Rules

- The assignment has a total of **10 marks**
- Work in your assigned teams of **up to 2 members**
- Submit your solutions **via Moodle**
- Solutions submitted after the deadline will automatically receive **0 marks**
- Copying solutions from or sharing with other students will lead to **exclusion from the course**
- The use of language software is **not permitted**; each group member must understand every part of the solution
- Document your solutions with **screenshots** and attach a **TEXT file** with your code/queries
- If anything is unclear, state your assumptions explicitly to make your solution understandable

### Task 1 5 Marks

For this task you will use **SparkContext** in PySpark. For more information about PySpark, read the documentation here: **Further Reading:**

▷ [PySpark API Documentation](#)

**Note:** Make sure to have Java  $\geq 1.8$  installed. Download the text file `Text-Data.txt` from Moodle, which is needed for tasks (c) and (d).

- (a) Create a list of 0 to 80,000, then use `parallelize()` to make an RDD from the list, distributed into **eight partitions**. Use the RDD's `filter()` and a lambda function to convert the list into a list of even numbers. Show the result using the `take()` function and output the first seven numbers of the even list.
- (b) Create an RDD with 40,000 random integers between 1 and 100, partitioned into **10 partitions**. Count how many numbers fall into each of the following buckets:

| Bucket | Range  |
|--------|--------|
| 1      | 1–10   |
| 2      | 11–20  |
| 3      | 21–30  |
| 4      | 31–40  |
| 5      | 41–50  |
| 6      | 51–60  |
| 7      | 61–70  |
| 8      | 71–80  |
| 9      | 81–90  |
| 10     | 91–100 |

Show the results as key-value pairs, e.g. `("1-10", count)`, `("11-20", count)`, ...

- (c) Explain what the `parallelize()` function does. Define RDD and describe the process used step-by-step in part (a). Afterwards, compare this approach with filtering odd numbers in a list using Python's built-in `filter()` function. Highlight the differences between using RDD's `filter()` and Python's `filter()` in terms of execution and scalability.
- (d) Count the number of appearances of each word in the text file `Text-Data.txt` using PySpark. Show the results of the 25 words with the most occurrences as a tuple, e.g. `("word", 23)`. Make sure to filter out any non-printable characters.  
**Hint:** You can use `flatMap()`, `filter()`, `reduceByKey()`, and lambda functions in PySpark/Python.
- (e) List all words which can be classified as **auxiliary verbs** or **function words** and exclude them from your results (e.g. "by", "to", "and" are non-meaningful words). Order each meaningful word by occurrence.

**Task 2** 5 Marks

This task focuses on querying the **E-Commerce** database in Neo4j using **Cypher**. Write one Cypher query for each subtask and briefly explain what your query does.

- (a) Write a Cypher query to retrieve all products in the category “Books” that satisfy the following conditions:

- Their average review rating is between 3 and 4 (inclusive).
- They have been purchased more than 5 times.

Return the product name, average rating, and total number of purchases. Sort the output by average rating in descending order.

- (b) Find all orders that contain more than 3 products.

Return the order ID and the number of products contained in each order. Sort the result by product count in descending order.

- (c) Retrieve the names of all products ordered by users who also ordered the product “Harbor Two”.

Each product name should appear only once in the output.

- (d) Retrieve all category names where the first letter of the category name lies between C and H (inclusive).

The output should contain the following columns:

| Category Name | Number of Products | Products List |
|---------------|--------------------|---------------|
|---------------|--------------------|---------------|

- (e) Begin by examining the properties of any existing **User** node in the database to understand the data structure. Then create a new **User** node for yourself using the same properties and fill them with your own information.

Once your user node has been created, simulate a realistic shopping journey for at least four products of your choice:

- View each product.
- Add each product to the cart.
- Place an order containing all selected products.
- Purchase each product.
- Write a review with a high rating for each purchased product.

*Hint:* Use **WITH** to chain operations. First inspect the properties of existing **User** and **Review** nodes, then create your user node and establish the relationships step by step following the natural order of the shopping journey.

Show your final result.

**Deliverables:** Submit a `.txt` file containing all Cypher queries and screenshots showing the execution results in Neo4j.

---

 End of Assignment 

---